

GESTIÓN DE SISTEMAS DE DATOS MASIVOS

Data Sources

01. Types of Data Sources

02. From unstructured data to actionable data

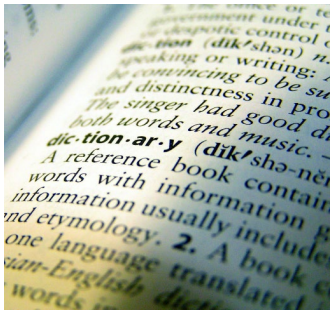
03. Information Retrieval Mechanisms

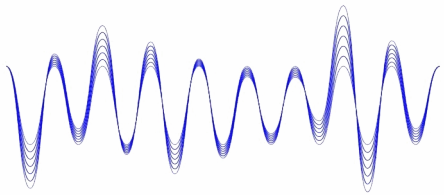
04. Information Preprocessing

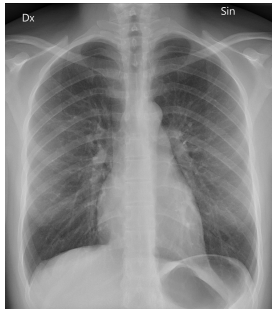
05. Information Transformation

Types of Data Sources

- **Based on their nature:**
  - **Words** : These sources use language, dictionaries, syntax, and semantics to express the data.
  - **Waves :** These sources use ordered numerical data to express the flow of data on a scale
  - **Image image**: These sources use images as a means of data transmission.







Types of Data Sources

- **Based on their form:**
  - **Structured** : Data that has an order and can be processed and interpreted uniquely.



  - **Unstructured :** Ambiguous data that requires an expert to process and can be interpreted in multiple ways.



Types of Data Sources

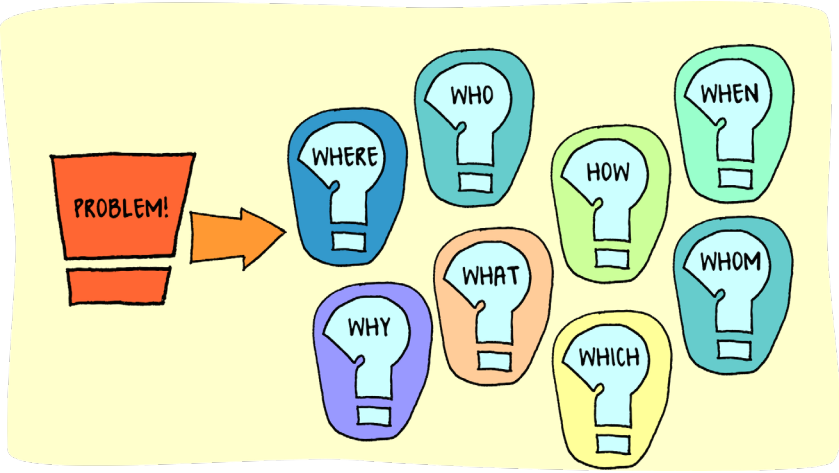
- **Based on their size:**
  - **Normal Data** : This information can be fully processed by current technology.
  - **Big Data :** This information needs to be reduced in its dimensionality to be processed by current technology.

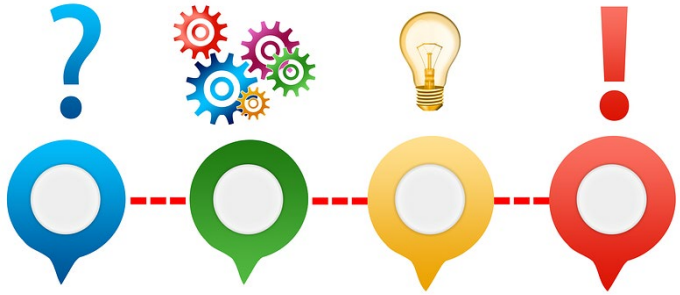




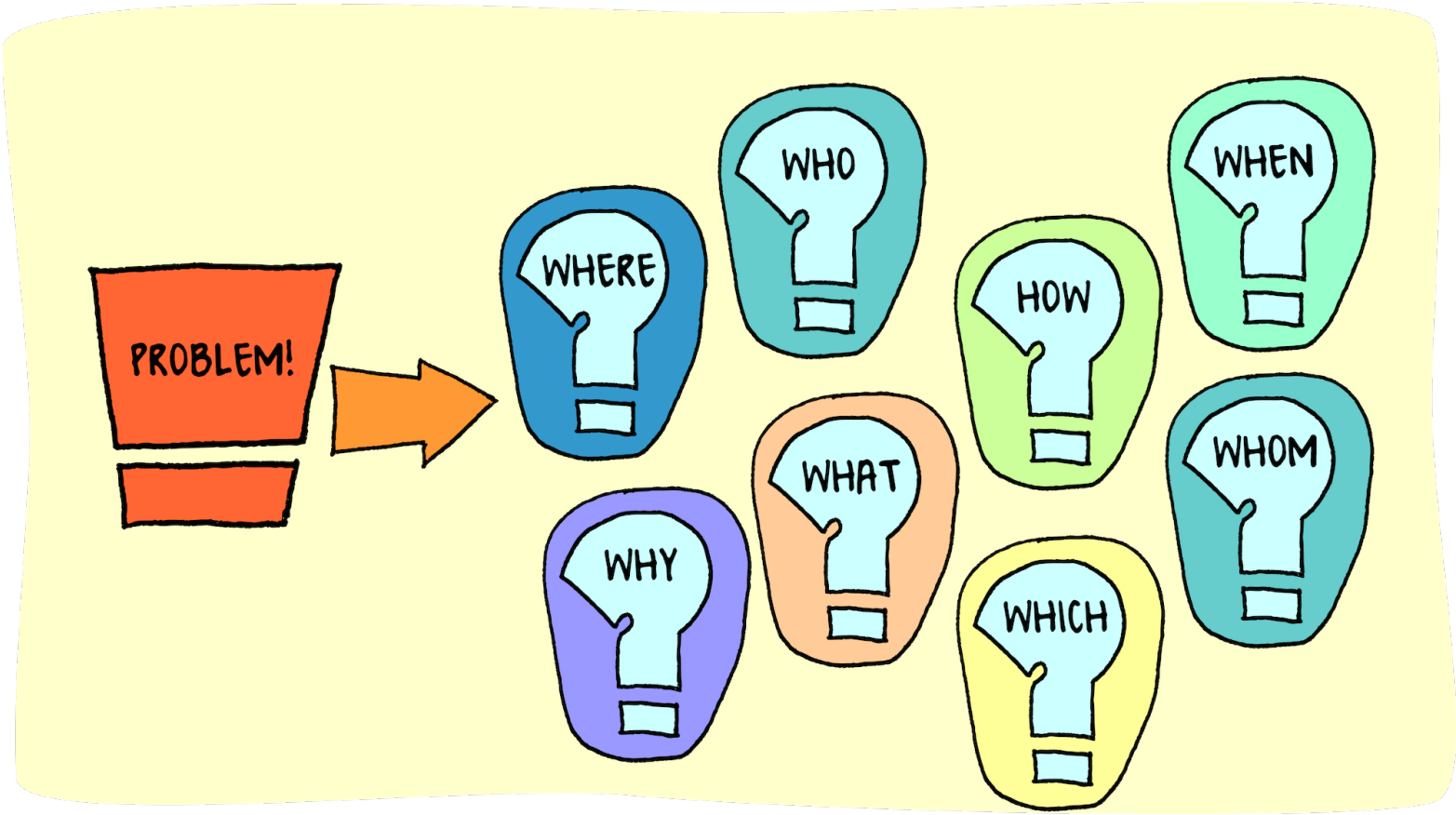
From unstructured data to actionable data

- **Problem**: Unstructured information requires prior study, and it must ensure that the interpretation is appropriate by the expert who created it..
- **"Solution"**: Structuring the information is often mandatory for processing, but there is always a degree of freedom to have misinterpreted the information by the expert or processing tool.

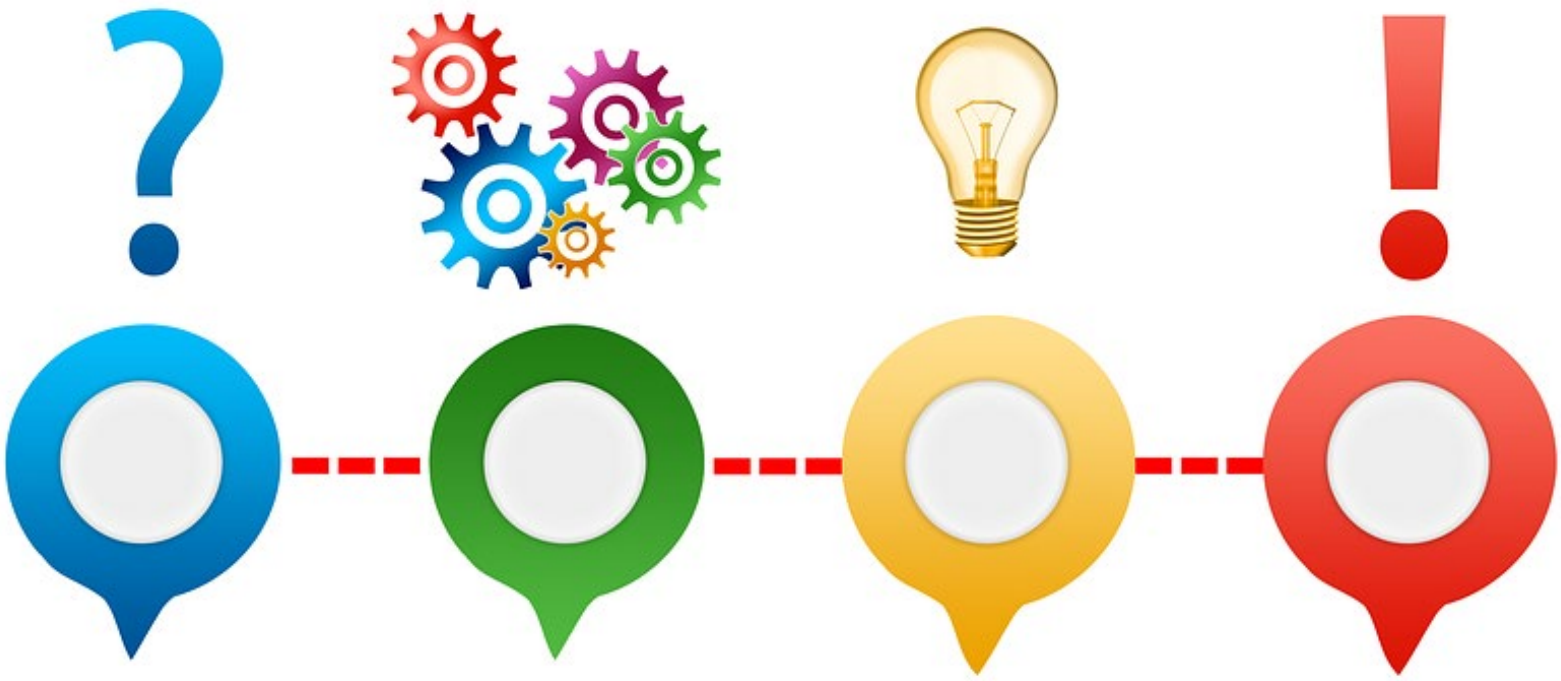




From unstructured data to actionable data

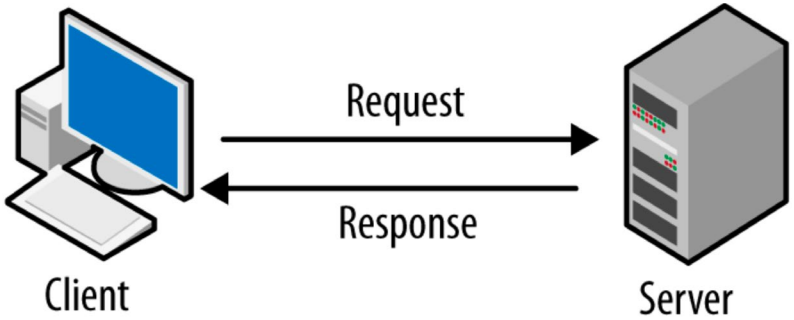


From unstructured data to actionable data



Information Retrieval Mechanisms

- **By service**:
  - Recovery through an online service that allows us to access information remotely.
  - This information is typically already structured and can be processed as is or reduced in case it is too voluminous or contains excess information not useful for processing.
  - It is highly recommended to store it locally in its raw form to avoid repeatedly accessing the service for the same information and potentially overloading the server that provides it.



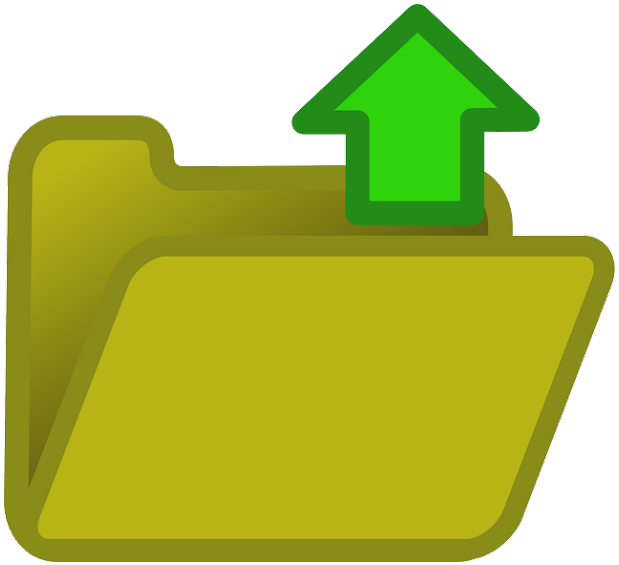
Information Retrieval Mechanisms

- **By service (Example)**:
  - *Web Scrapping* : Web scraping refers to the process of extracting content and data from websites using software. It is mixed with unstructured information, the text from the web page.



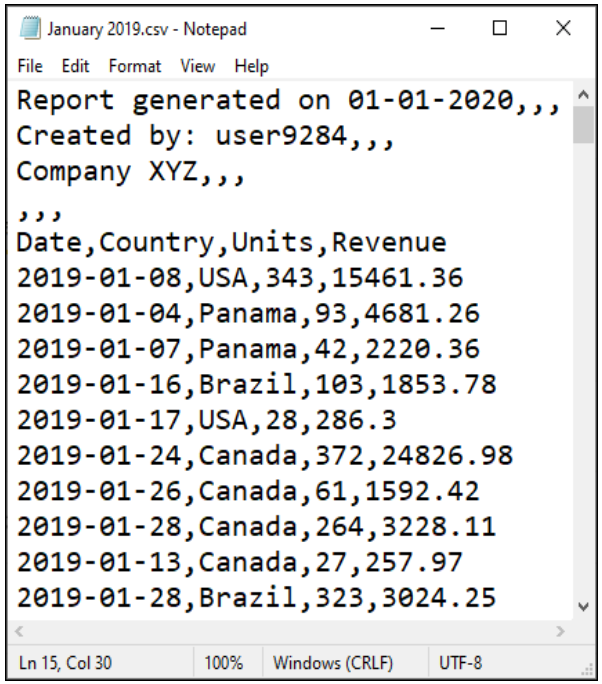
Information Retrieval Mechanisms

- **By File**:
  - Web scraping refers to the process of extracting content and data from websites using software. It is mixed with unstructured information, the text from the web page.



Information Retrieval Mechanisms

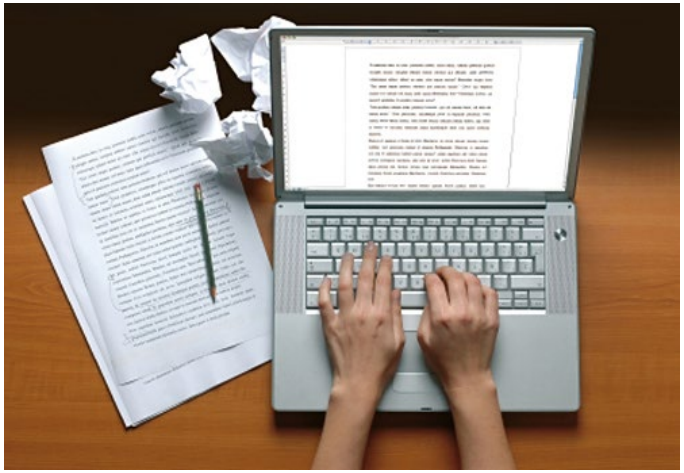
- **By File (CSV) :**
  - CSV is a basic structured file format where the meaning of categories is in the first row, and each row is a data record.



From: learn.microsoft.com

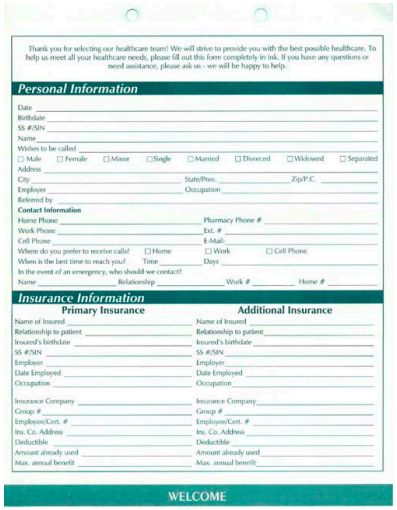
Information Retrieval Mechanisms

- **Manually**:
  - Information can be manually entered into the destination system by the expert. This process is the slowest and requires the expert to input the data into the system themselves.
  - This mechanism ensures the quality of the information and also allows for data anonymization and compliance with data privacy regulations.



Information Retrieval Mechanisms

- **Manually (medical data)**:
  - Medical reports are quite challenging to process, involving issues of security, anonymization, and localization. Typically, in medical information retrieval processes, one must physically go to the healthcare facility and retrieve the already processed (and anonymized) information to take it out of the building. Due to this, they often lack compatible formats and highly efficient retrieval services.


Thank you for selecting our healthcare team! We will strive to provide you with the best possible healthcare. To help us meet all your healthcare needs, please fill out this form completely in ink. If you have any questions or need assistance, please ask us - we will be happy to help.

Personal Information

Date
Birthdate
SS #/SIN
Name
Wishes to be called
☐ Male ☐ Female ☐ Minor ☐ Single ☐ Married ☐ Divorced ☐ Widowed ☐ Separated
Address
City State/Prov. Zip/P.C.
Employer Occupation
Referred by
Contact Information
Home Phone Pharmacy Phone #
Work Phone Ext. #
Cell Phone E-Mail
Where do you prefer to receive calls? ☐ Home ☐ Work ☐ Cell Phone
When is the best time to reach you? Time Days
In the event of an emergency, who should we contact?
Name Relationship Work # Home #

Insurance Information

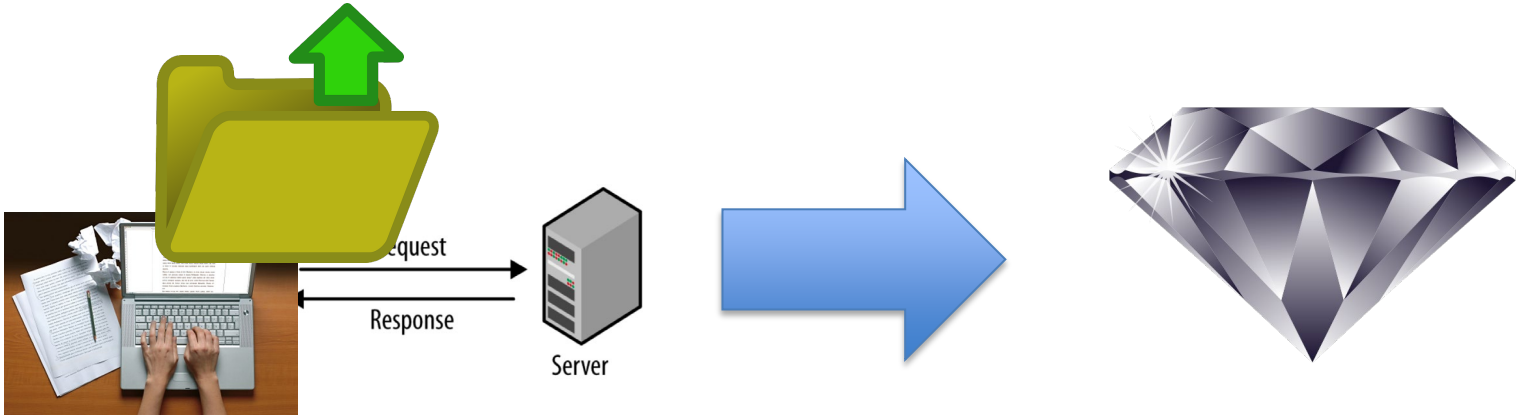
Primary Insurance	Additional Insurance
Name of Insured	Name of Insured
Relationship to patient	Relationship to patient
Insured's birthdate	Insured's birthdate
SS #/SIN	SS #/SIN
Employer	Employer
Date Employed	Date Employed
Occupation	Occupation
Insurance Company	Insurance Company
Group #	Group #
Employee/Cert. #	Employee/Cert. #
Ins. Co. Address	Ins. Co. Address
Deductible	Deductible
Amount already used	Amount already used
Max. annual benefit	Max. annual benefit

WELCOME

From: www.medical-forms.com

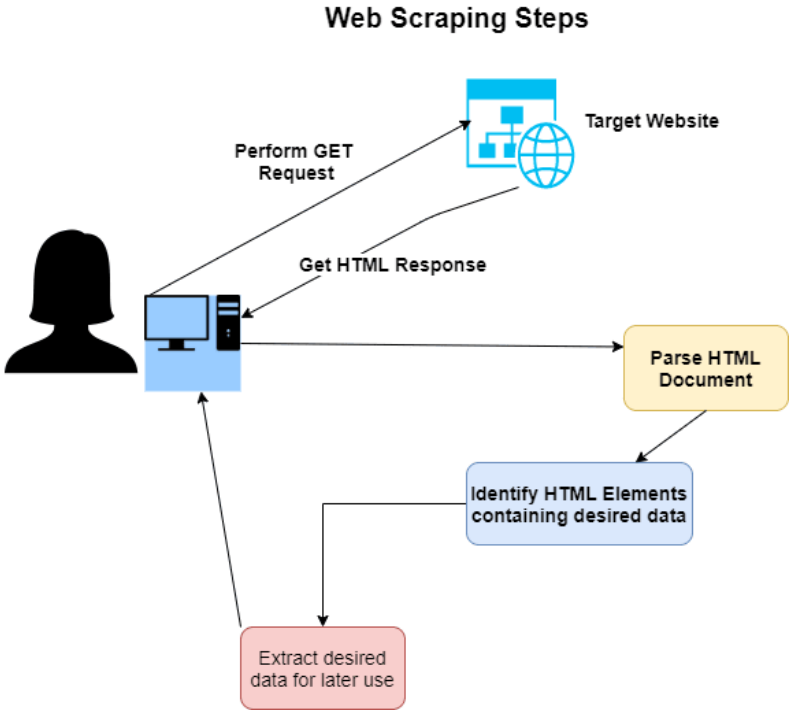
Information Retrieval Mechanisms

- **Combination (part, of all, of them)**:
  - In most cases, a combination of multiple information retrieval techniques will be necessary for obtaining data for analysis, along with various processing and transformation mechanisms to unify the information into a single source of homogeneous data..



Web Scrapping

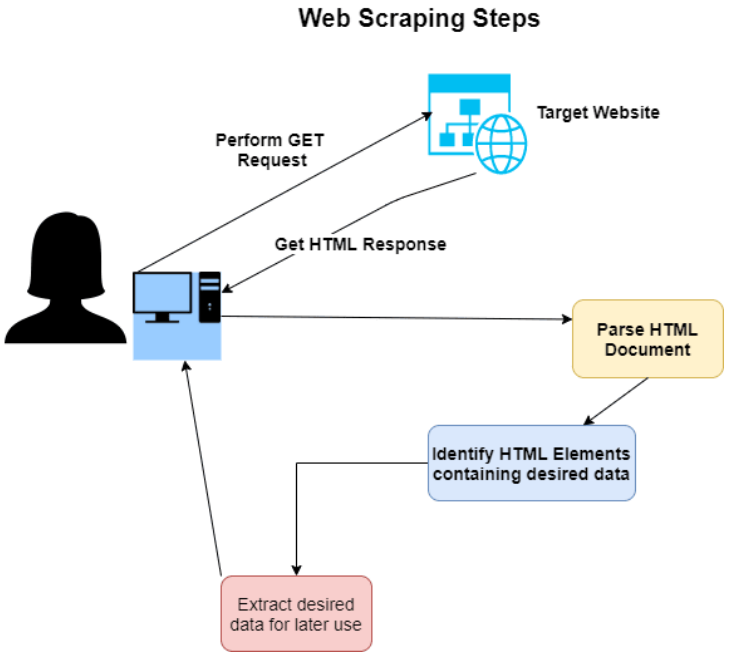
- **Collect Data**: This phase involves the retrieval of information from the web page in static HTML format. This process may require complex browser simulation techniques or session access if the website has security or high interactivity.
- **Store Data:** Until there is confirmation that the process has been completed successfully, it is highly recommended to store the web in HTML or XML format to avoid making repeated calls to the web for its retrieval. This factor is crucial on certain websites, as the web scraping process can be seen as a DDoS attack.



From: dev.to

Web Scrapping

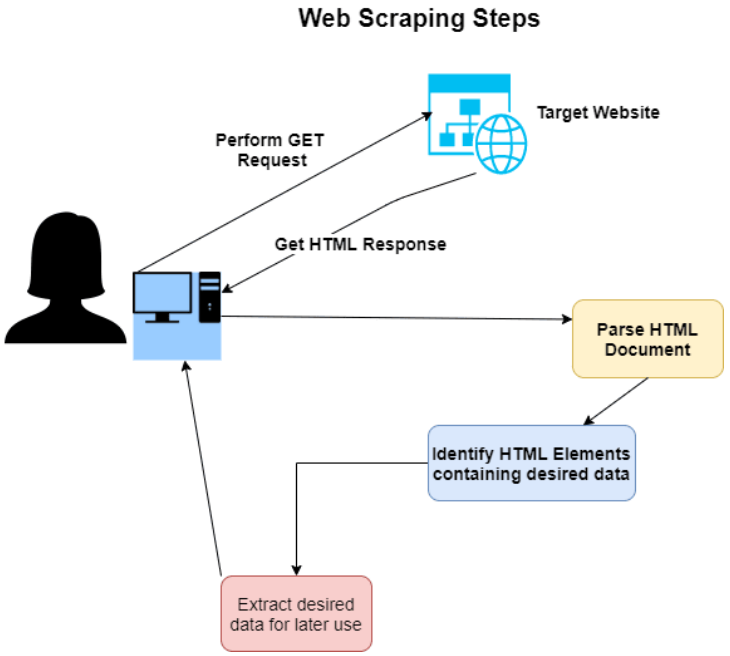
- **Process Data**: Indeed, an HTML page can be seen as a document very similar to XML, so it can be processed using its tags to reduce the information coming from HTML into a smaller entity that contains only valuable information for the analysis process. This process, in turn, is divided into two phases:
  - *Identification of valuable elements*: Document analysis to retrieve those parts that can be useful in the analysis.
  - Information processing: Information processing for the creation of valuable material and evaluation of new complementary information sources.



From: dev.to

Web Scrapping

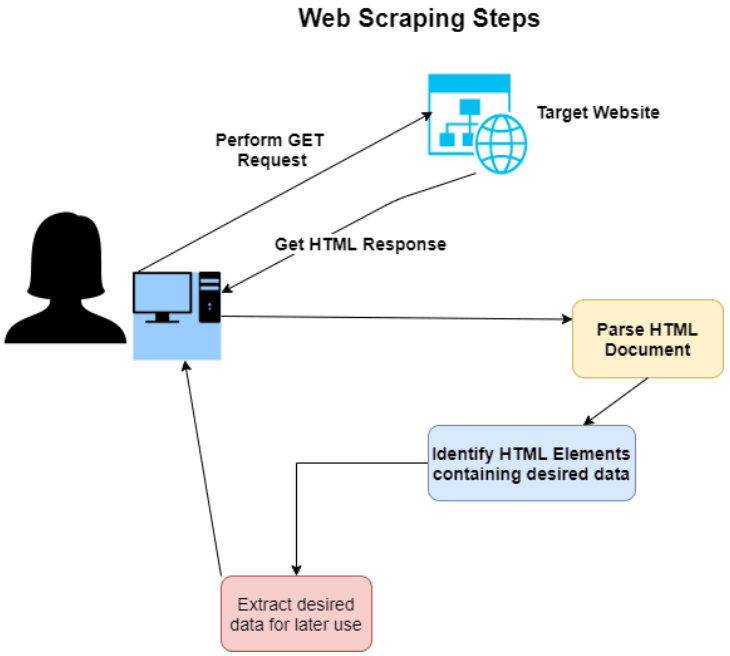
- **Pos-Procesing Data**: The complementary information can once again be one or more web pages, so the scraping steps are repeated.



From: dev.to

Web Scrapping

- ***Notes***
  - Storing the web when retrieving it serves two main purposes: firstly, it prevents the need to repeatedly retrieve it during script creation processes; secondly, it allows for halting when encountering a loop, where one web references another already processed, which can potentially lead to an infinite loop.
  - It's essential to define a scraping depth limit. Potentially, if one website is complemented by others, the completion process can be infinite, with the web being scraped complemented by other webs until a closed loop is formed, which is not always guaranteed on the internet. Therefore, it's crucial to always establish a maximum scraping depth.
  - Scraping requires prior knowledge of the web's structure and its peculiarities for the proper retrieval and processing of information. This process falls within the initial analysis phase, where data sources are evaluated.



From: dev.to

Web Scrapping (python)

- ***Library "request"***
  - The 'request' library is a simple method for retrieving a URL. This library makes a GET call to the provided URI and retrieves the HTML it returns from the target.
  - This type of retrieval is the simplest and is recommended for static websites where the retrieval process involves getting the HTML, reviewing links, and obtaining related links.

```
import requests

x = requests.get('https://w3schools.com/python/demopage.htm')

print(x.text)
```

From: www.w3schools.com

Web Scrapping (python)

- ***Library "BeautifulSoup"***
  - This library is an HTML parser, and by extension, it works with similar formats like XML and XHTML, allowing for easy and intuitive document retrieval and exploration.
  - By using commands, you can make queries and retrieve elements from the document, enabling iteration for the purpose of obtaining or processing parts of it.

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html_doc, 'html.parser')
print(soup.prettify())
```

From: https://www.crummy.com/software/BeautifulSoup/bs4/doc/

Web Scrapping (python)

- ***Library "BeautifulSoup" (retrieve elements)***
  - When we have the document parsed with Beautiful Soup, we can perform element analysis to retrieve parts of it using basic criteria and the use of the find() function.
  - This function allows us to search by section (e.g., 'p'), by class (e.g., 'class="important_reg"'), by identifier, by value... in essence, it enables the exploration of the consulted document's tree of attributes that make up the elements on the HTML page and their values.



Web Scrapping (python)

- ***Library "BeautifulSoup" (retrieve elements by type)***

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html_doc, 'html.parser')

soup.find_all('a')

//////////////////
 [<a href="https://example1.com">Example 1</a>,
<a href="https://example2.com">Example 2</a>,
<a href="https://example3.com">Example 3</a>]
```

Web Scrapping (python)

- ***Library "BeautifulSoup" (retrieve elements by class)***

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html_doc, 'html.parser')

soup.find_all(class_='inline')

//////////////////
 [<span class="inline"><p>This is span 1</p></span>,
 <span class="inline"><p>This is span 2</p></span>,
 <span class="inline"><p>This is span 2</p></span>]
```

Web Scrapping (python)

- ***Library "BeautifulSoup" (retrieve elements by type and class)***

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html_doc, 'html.parser')

soup.find_all('div', class_='footer')

//////////////////
[<div class="footer">
    This is the footer
    <span class="inline"><p>This is span 1</p></span>
<span class="inline"><p>This is span 2</p></span>
<span class="inline"><p>This is span 2</p></span>
</div>]
```

Web Scrapping (python)

- ***Library "BeautifulSoup" (retrieve elements by another attribute diferents as class)***

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html_doc, 'html.parser')

soup.find_all(type='footer')

//////////////////
[<input id="POST-name" name="name" type="text"/>]
```

Web Scrapping (python)

- ***Library "BeautifulSoup" (retrieve elements by contained text inside element)***

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html_doc, 'html.parser')

soup.find_all('h2', string='Formulario')

//////////////////
 [<h2>Formulario</h2>]
```

Web Scrapping (python)

- ***Library "BeautifulSoup" (retrieve elements by regular expresion)***

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html_doc, 'html.parser')

soup.find_all(re.compile (r'^h\d+.*'))

//////////////////
[<h1>Just testing</h1>, <h2>Some links</h2>, <h2>Formulario</h2>]
```

Web Scrapping (python)

- ***Library "BeautifulSoup" (retrieve elements by combination of criteria)***

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html_doc, 'html.parser')

soup.find_all(['input', 'span'])

//////////////////
[<input id="POST-name" name="name" type="text"/>,
 <input type="submit" value="Save"/>,
 <span class="inline"><p>This is span 1</p></span>,
 <span class="inline"><p>This is span 2</p></span>,
 <span class="inline"><p>This is span 2</p></span>]
```

Web Scrapping (python)

- ***Library "BeautifulSoup" (retrieve elements by parental)***

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html_doc, 'html.parser')

gold = soup.find('li', 'gold')
gold.find_parents('div')

////////
[<div class="block"> <h2>Some links</h2> <p>Hi there!</p> <ul id="data"> <li
class="blue"><a href="https://example1.com">Example 1</a></li> <li class="red"><a
href="https://example2.com">Example 2</a></li> <li class="gold"><a
href="https://example3.com">Example 3</a></li> </ul> </div>]
```

Web Scrapping (python)

- ***Library "BeautifulSoup" (nature and attributes of element)***

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html_doc, 'html.parser')

#Name retrieval
gold = soup.find('li', 'gold')
gold.name
///////
'li'

gold['id']
///////
'1234'
```

```
[<div class="block">
        <h2>Some links</h2>
         <p>Hi there!</p>
         <ul id="data">
                <li class="blue"><a
href="https://example1.com">Example
1</a></li>
                 <li class="red"><a
href="https://example2.com">Example
2</a></li>
                 <li class="gold id=1234 "><a
href="https://example3.com">Example
3</a></li>
        </ul>
 </div>]
```

Web Scrapping (python)

- ***Library "lxml"***
  - This library is necessary for searching elements by XPATH. This type of search is very convenient when classes and IDs are not distinctive, and it's the position in the HTML tree that determines the meaning. Modern browsers often have built-in methods in their developer panels to quickly obtain the XPATH of an element, which is why it's widely used by many scraping users.

```
from bs4 import BeautifulSoup
from lxml import etree
import requests

URL = "https://en.wikipedia.org/wiki/Nike,_Inc."

HEADERS = ({'User-Agent':
            'Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 \
            (KHTML, like Gecko) Chrome/44.0.2403.157 Safari/537.36',\
            'Accept-Language': 'en-US, en;q=0.5'})

webpage = requests.get(URL, headers=HEADERS)
soup = BeautifulSoup(webpage.content, "html.parser")
dom = etree.HTML(str(soup))
print(dom.xpath('//*[@id="firstHeading"]')[0].text)
```

From: https://www.geeksforgee

Web Scrapping (python)

- ***Library "Selenium"***
  - This type of retrieval simulates or uses a browser already installed on the system to navigate. This implies that it requires a browser with a compatible version or the installation of virtual browsers like Selenium.
  - The navigation process is real. If the system's browser is used, it will visually perform the navigation process and execute dynamic elements in the browser. If it's virtual, this process will be transparent to the user but will be performed in the same way.
  - It carries the risk of code execution without supervision or visual blocking of the navigation if it takes control of the browser itself.
  - As an advantage, by executing dynamic content, we can view the page just as a real user would. It also allows navigation by clicking, enabling real web browsing.

Web Scrapping (python)

- ***Library "Selenium" (Continuing)***
  - It allows the capture of HTML at any point in the navigation, allowing navigation on the same website but with different configurations by clicking on dynamic content.
  - As a trade-off, if it's running in hidden mode, it's more complex to analyze the state of the web. On the other hand, if it runs on the system's own browser, we'll see the process on the screen, and we won't be able to interact unless we want to affect the retrieval outside of the code's control.
  - It also allows the use of passwords, application keys, or navigation cookies that enable access to websites with increased security. Nevertheless, some pages can detect the process and may block navigation for security reasons.

Web Scrapping (python)

- ***Library "Selenium" (use)***
  - Installation : https://selenium-python.readthedocs.io/installation.html#
  - Ejemplos : https://aprendepython.es/pypi/scraping/selenium/

```
from selenium import webdriver
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.common.by import By

driver = webdriver.Firefox()
driver.get("http://www.python.org")
assert "Python" in driver.title
elem = driver.find_element(By.NAME, "q")
elem.clear() elem.send_keys("pycon")
elem.send_keys(Keys.RETURN)
assert "No results found." not in driver.page_source
driver.close()
```

From: https://selenium-python.readthe

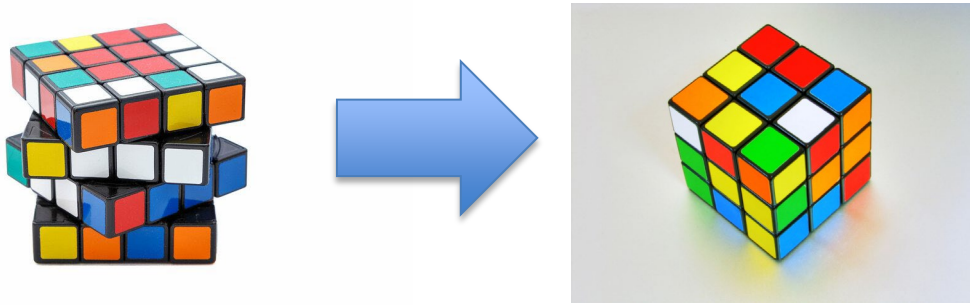
Web Scrapping (python)

- ***Library "Selenium" (use)***
  - Selenium, in turn, has its own system for searching by attributes, classes, regular expressions, and XPath, like the Beautiful Soup system.
  - At any moment, just like when using 'request', you can obtain the current HTML and pass it to Beautiful Soup for processing.

```
…
driver.get("http://www.python.org")
assert "Python" in driver.title
elem = driver.find_element(By.NAME, "q")
elem2 = driver. find_element_by_css_selector("h4")
elem3 = driver. find_element_by_id("1234")
elem4 = driver. find_element_by_class("text-important")
…
```

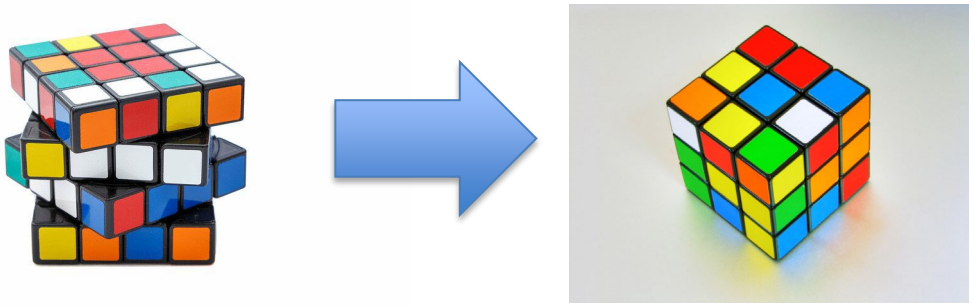
Information Preprocessing

- When working with external data (non-manual) or with multiple datasets from different sources, it's reasonable for them to not contain the information in the same syntax, semantics, or even scale/dictionary.
- Hence, preprocessing of the information is required before it can be considered reliable data.



Information Preprocessing

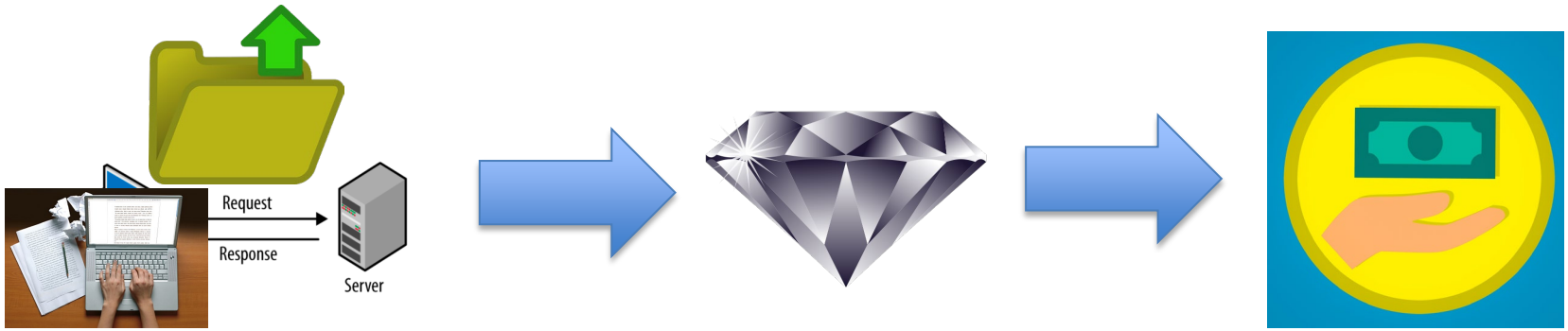
- The expected problems in this scenario are usually the following:
  - The data shares a structure, but its dictionary/scale is not the same.
    - They require a simple processing of equivalence or transformation to be combined.
  - The data shares a dictionary/scale but not structure.
    - They require agreeing on a compatible common structure that allows accommodation for both structures and their data.
  - The data does not share semantics.
    - They require a complex processing to accommodate a common semantics.
  - Combination of the previous cases.
    - It requires a combination of solutions to address them (worst-case and most common scenario).



Information Transformation

- In certain scenarios, the use of transformation mechanisms is required, not only to adapt the model and harmonize it but also to produce information that is consumable by an expert analysis system, the destination system for information, or, in general, as an intermediate step to adapt to a system outside of our control.

- Viewed in an extreme manner, the actual process of analysis and presentation of results is, in itself, a transformation of the data, a byproduct of the data itself, so that it can be a source of analysis once again. In most cases, it involves manual analysis by the domain expert who will make decisions based on the analysis results.





POLITÉCNICA

UNIVERSIDAD
POLITÉCNICA
DE MADRID



Universidad
Politécnica
de Madrid

ETSI **SISTEMAS INFORMÁTICOS**